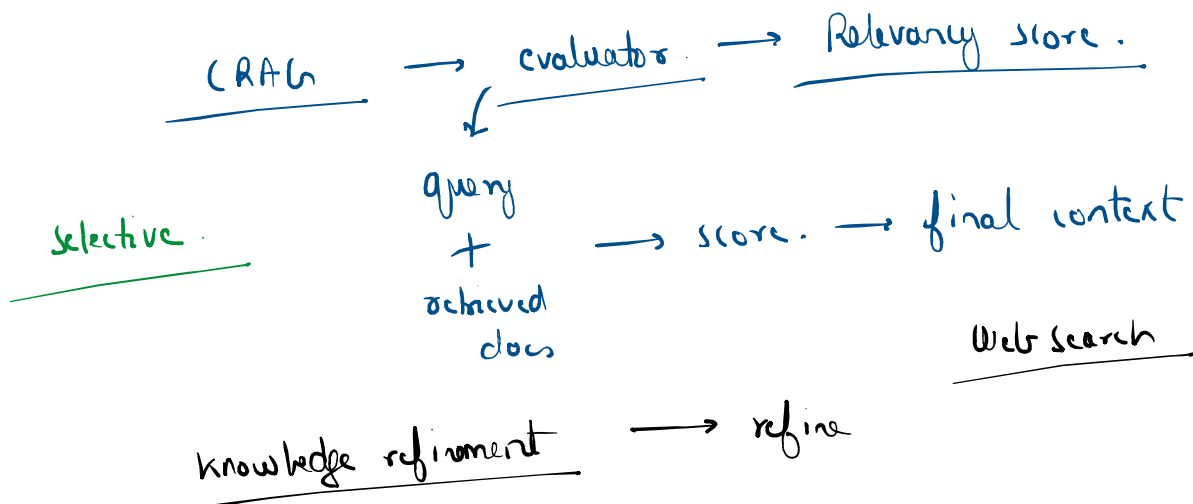
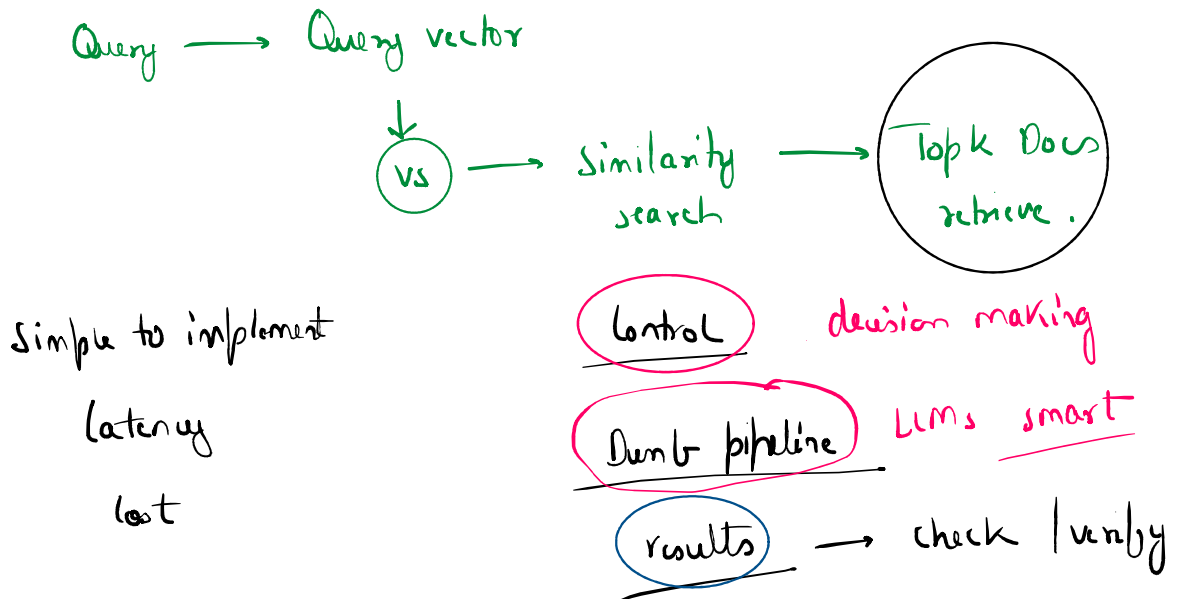


Traditional RAG



Self RAG → Self Reflective RAG.

Query

smart approach
decision making

↓
LLM

- ① Whether we need retrieval?
- ② ✓ Relevancy of retrieved docs?
- ③ Response → grounded / not?
- ④ Whether response → query?
↓
useful

Order
does
not
change

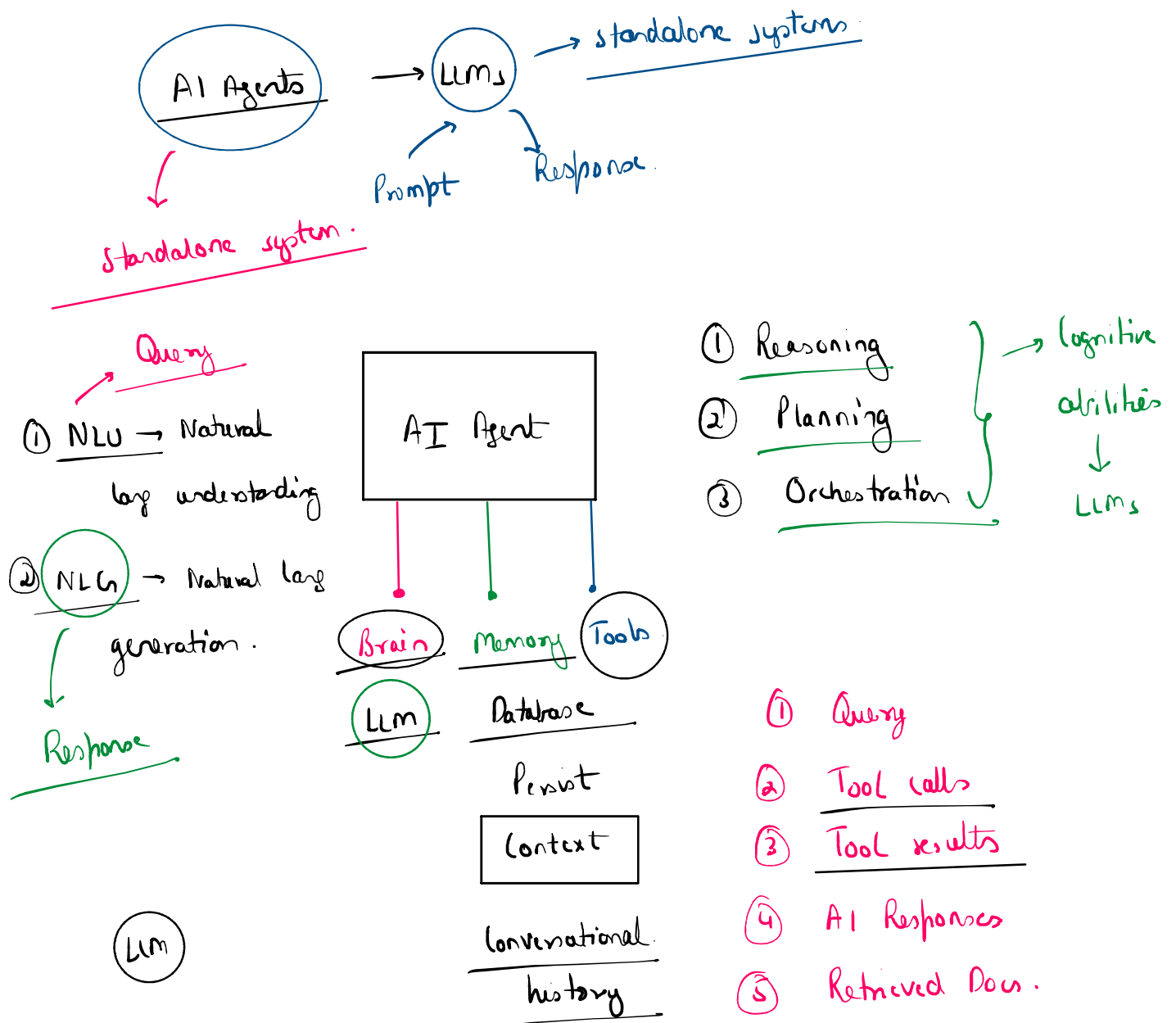
Agentic RAG → upgrade
↓
[AI Agents]

Evaluator → LLM.

Decision → LLM

- ① Autonomous → decision making → autonomous
- ② Adaptive → Retry, Try multiple sources,
knowledge refinement.
- ③ Dynamic → Breakdown, Rewrite.
↓
generation prompt → Tweak.

What are AI Agents?



Properties of AI Agents :-

① Autonomous / Autonomy → Decision Making.

② Planning → Series of steps - Actions
Roadmap

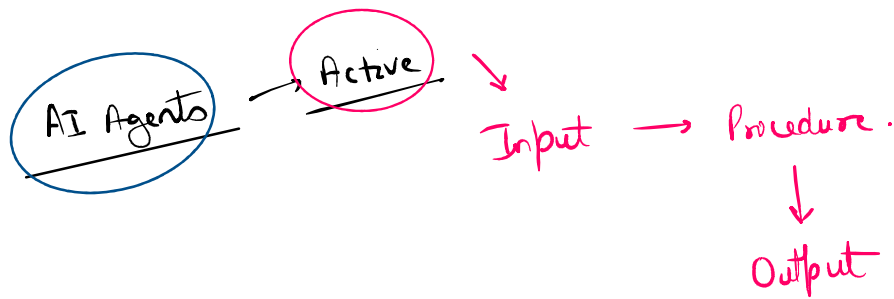
③ Reasoning $\rightarrow$ Thinking $\rightarrow$ Execution of each step

Now

Query $\rightarrow$ Ambiguous
 $\downarrow$
Rewrite / Multiple Queries

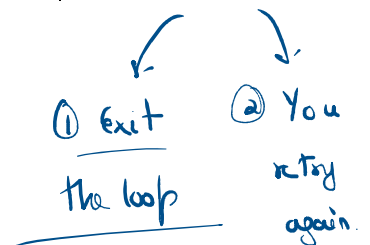
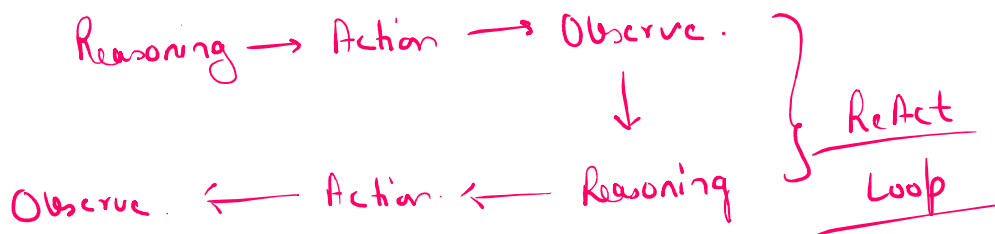
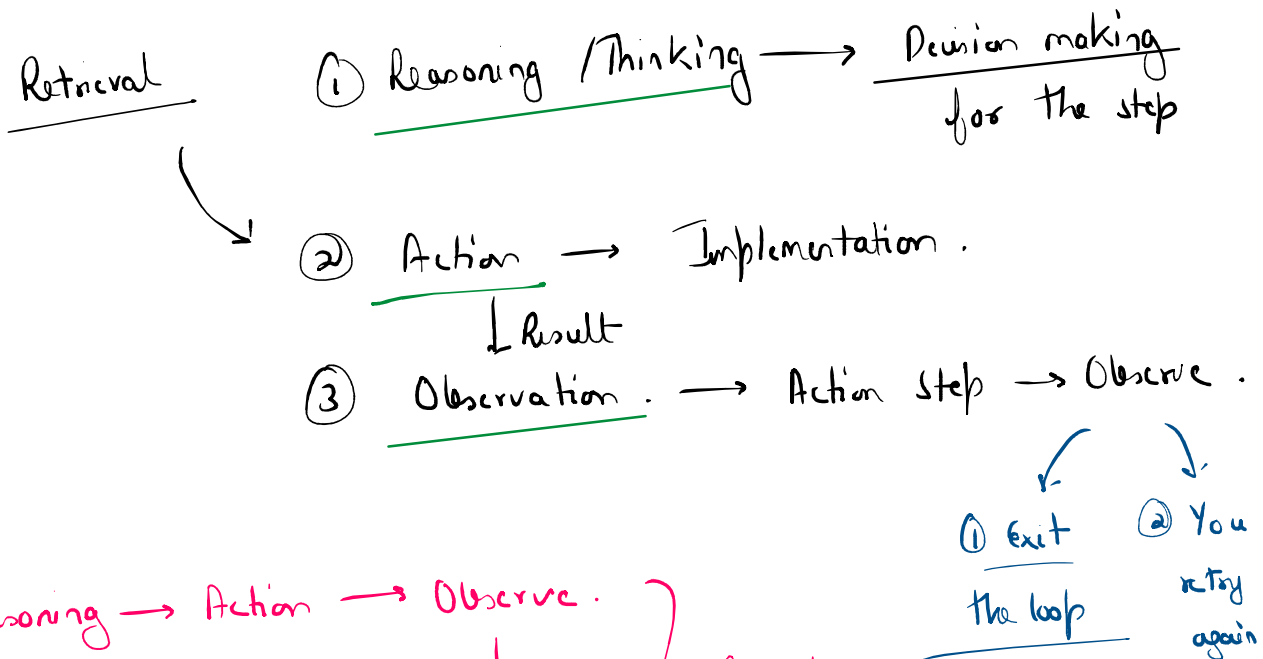
④ Orchestration → Planning / Reasoning

⑤ Adaptability → Dynamic, Robust, Error free.



ReAct Loop

Re → Reasoning Reasoning + Action.
 Act → Action.

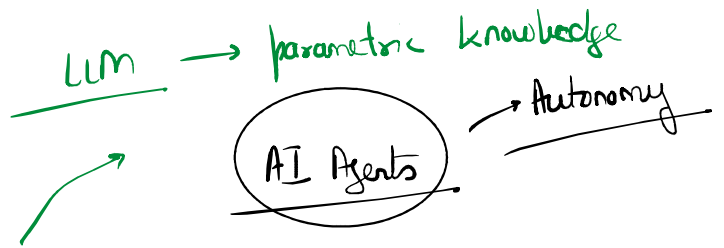


Traditional RAG

- ① Control on retrieval. → Reason
- ② Order is static → Action → dumb approach.
- ③ No verification of results.

ReAct Loop

Agentic RAG



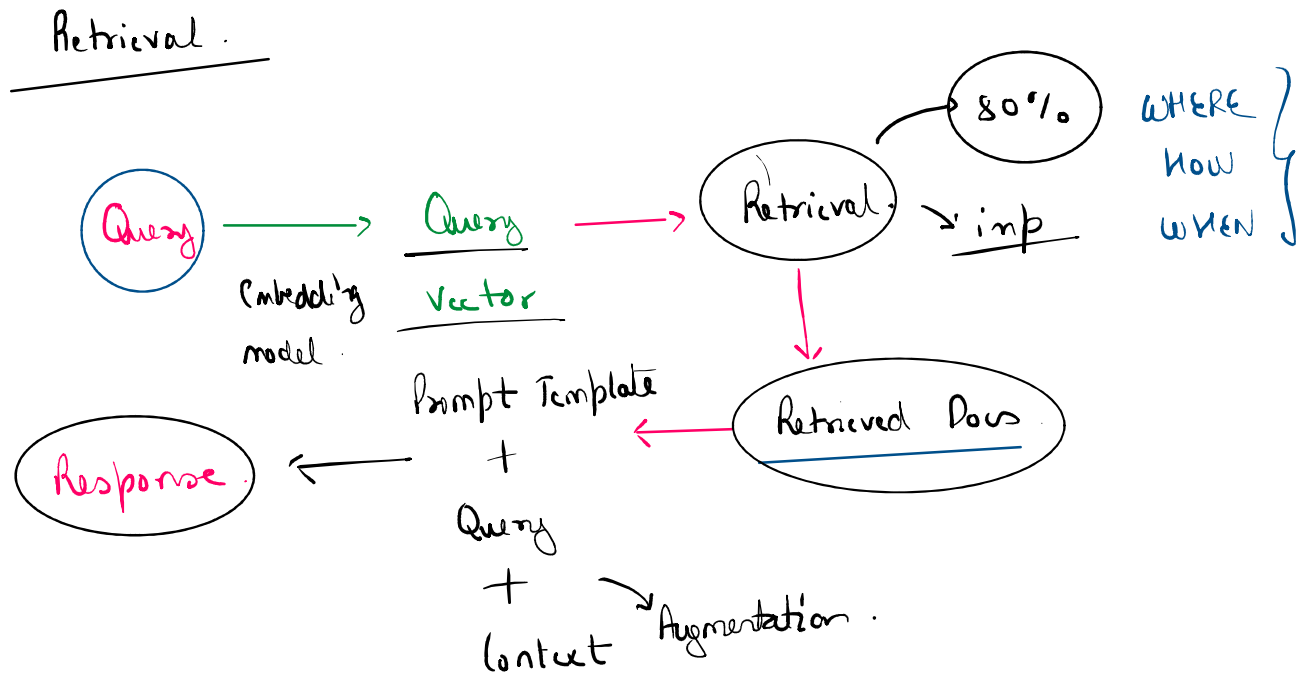
IF → Whether to do retrieval or not

WHERE → Where to do retrieval from. (tools) → Similarity Search

How → How to do it (tools)

k → , metadata filter → fields

WHEN → Dynamic workflows

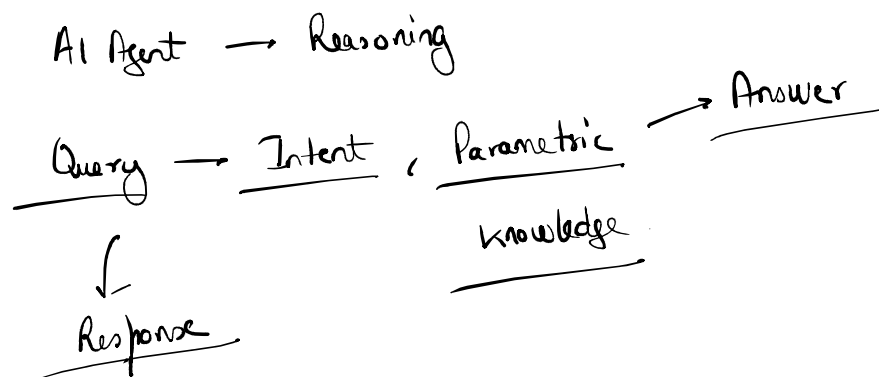


① Query → Input

① Check for retrieval.

② Rewrite / Rephrase.

① whether retrieval is required or not? ③ Query decomposition.



② Query is ambiguous.

↓
Rewrite. , Multiple versions.

③ Query is complex

Compare Python and JS and give me a comparison report

Decompose or break down

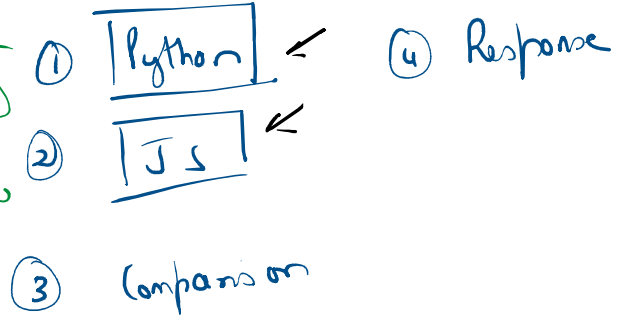
① Planning

② Reasoning

How

Multiple

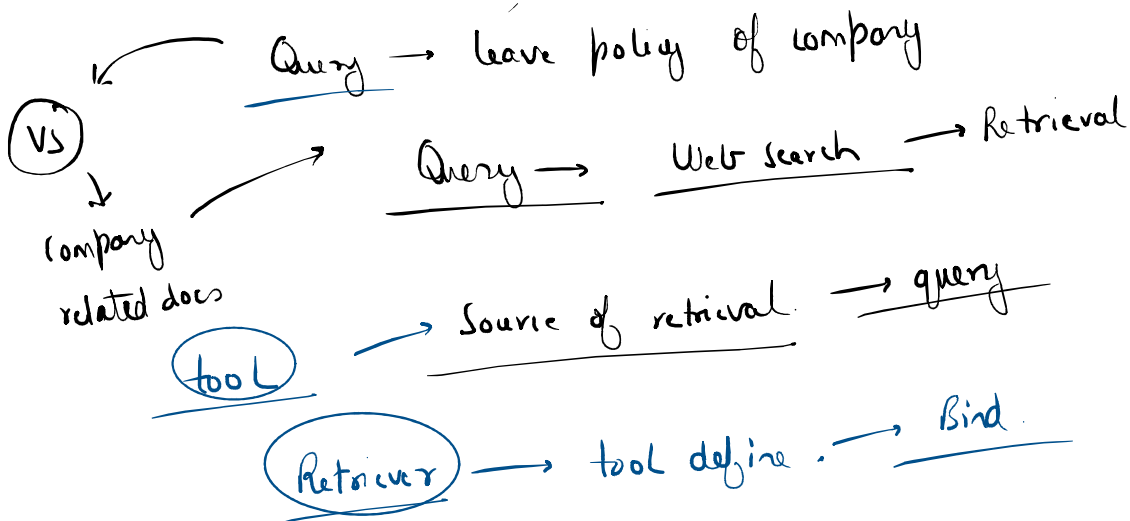
Retrievers



WHERE

Source of Retrieval → dynamically

Retry logic



How

Retrieval from selected source.

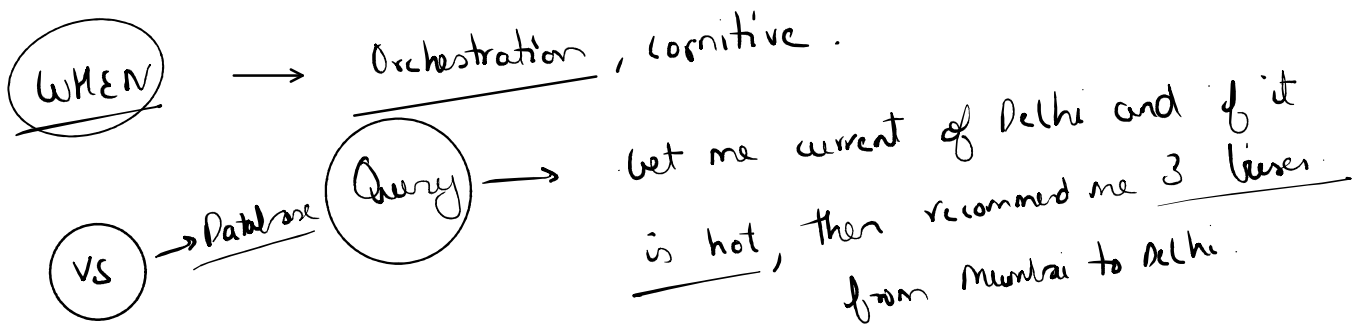
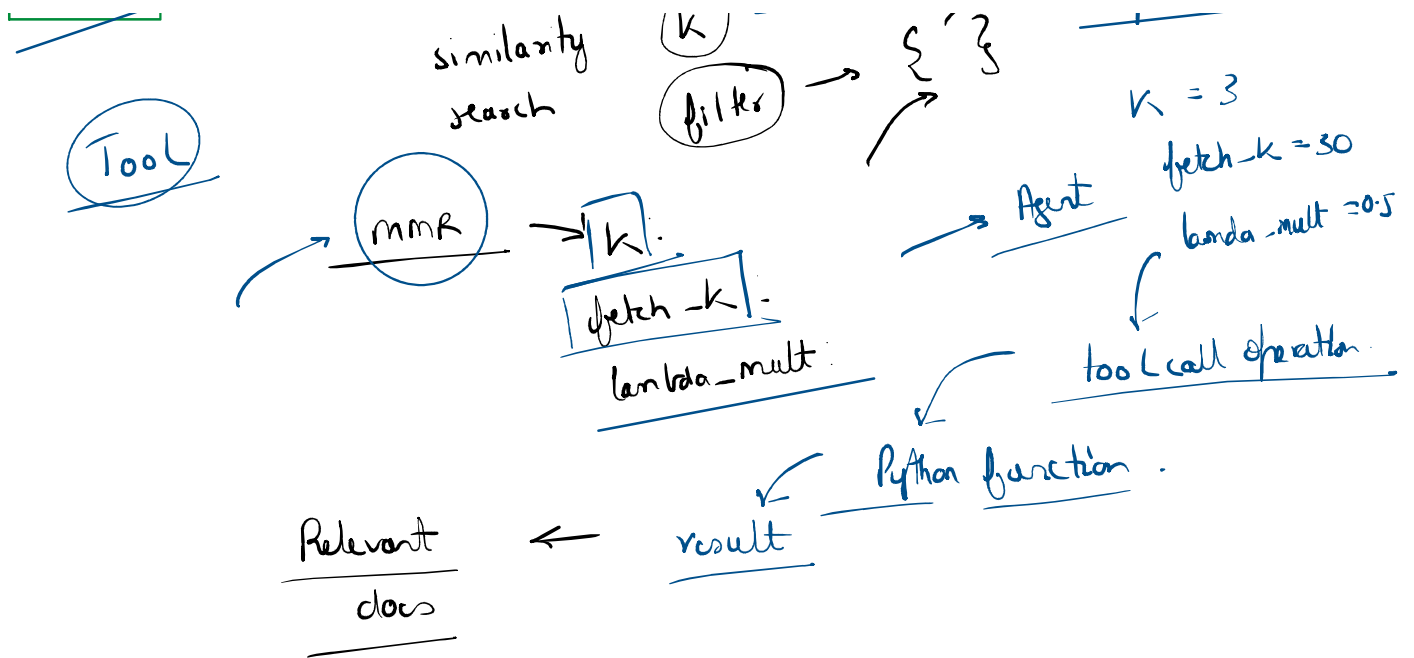
similarity



Tool schema

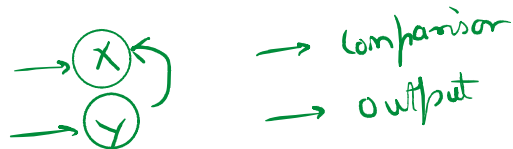
parameters

$k = 3$



Decompose

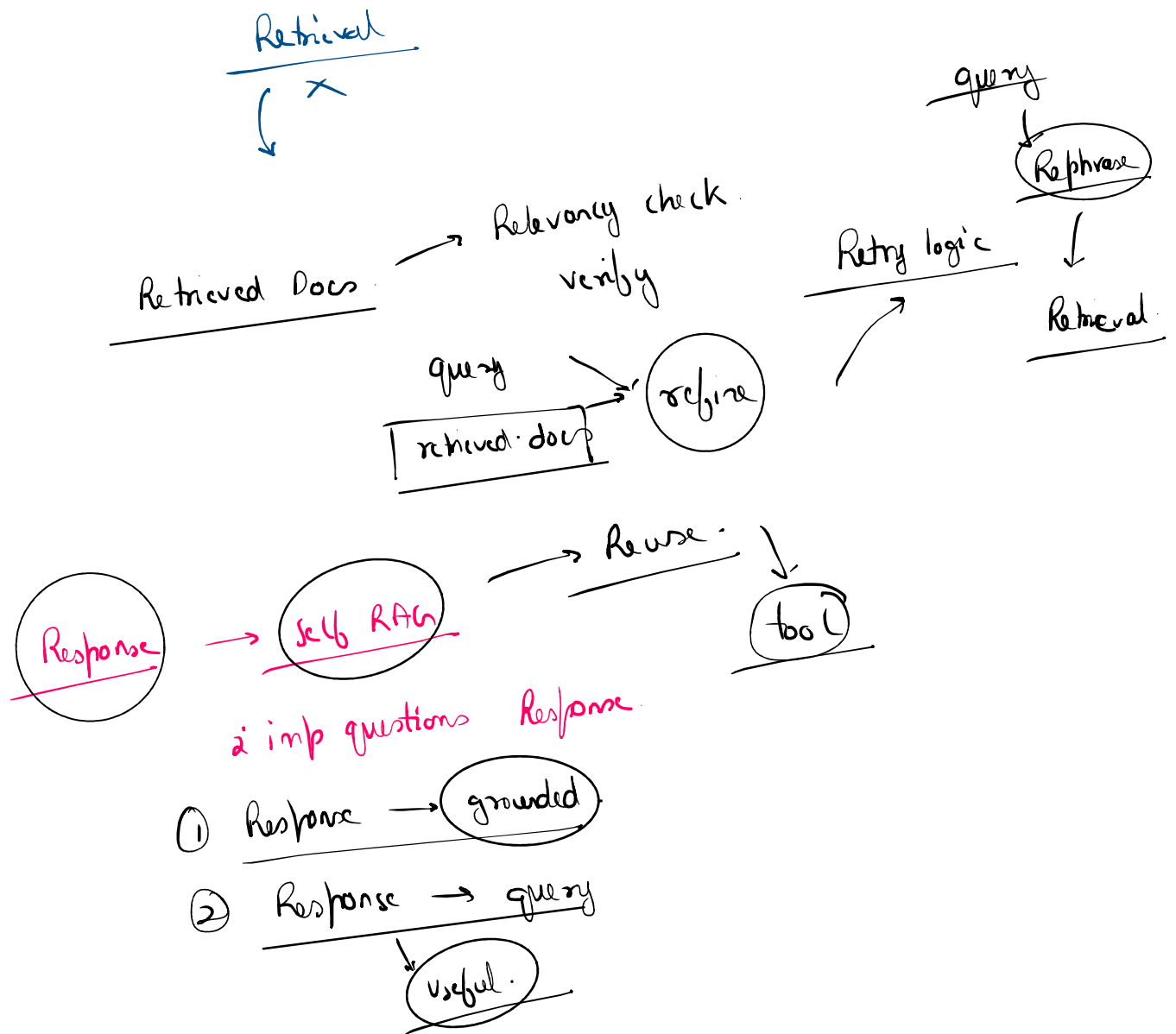
Query → Do a comparison b/w X and Y and show me the comp report



Retroy logic → Multiple Sources

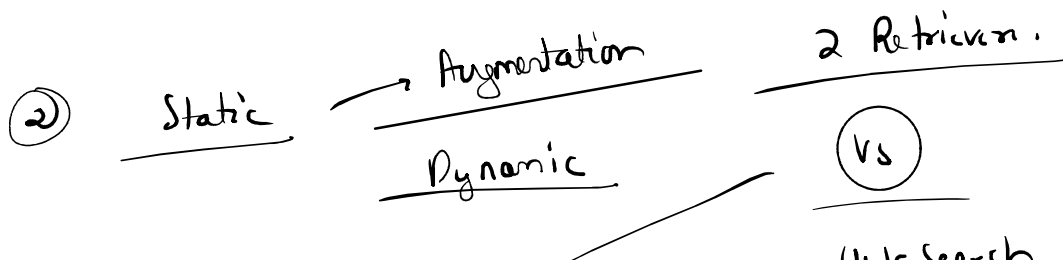
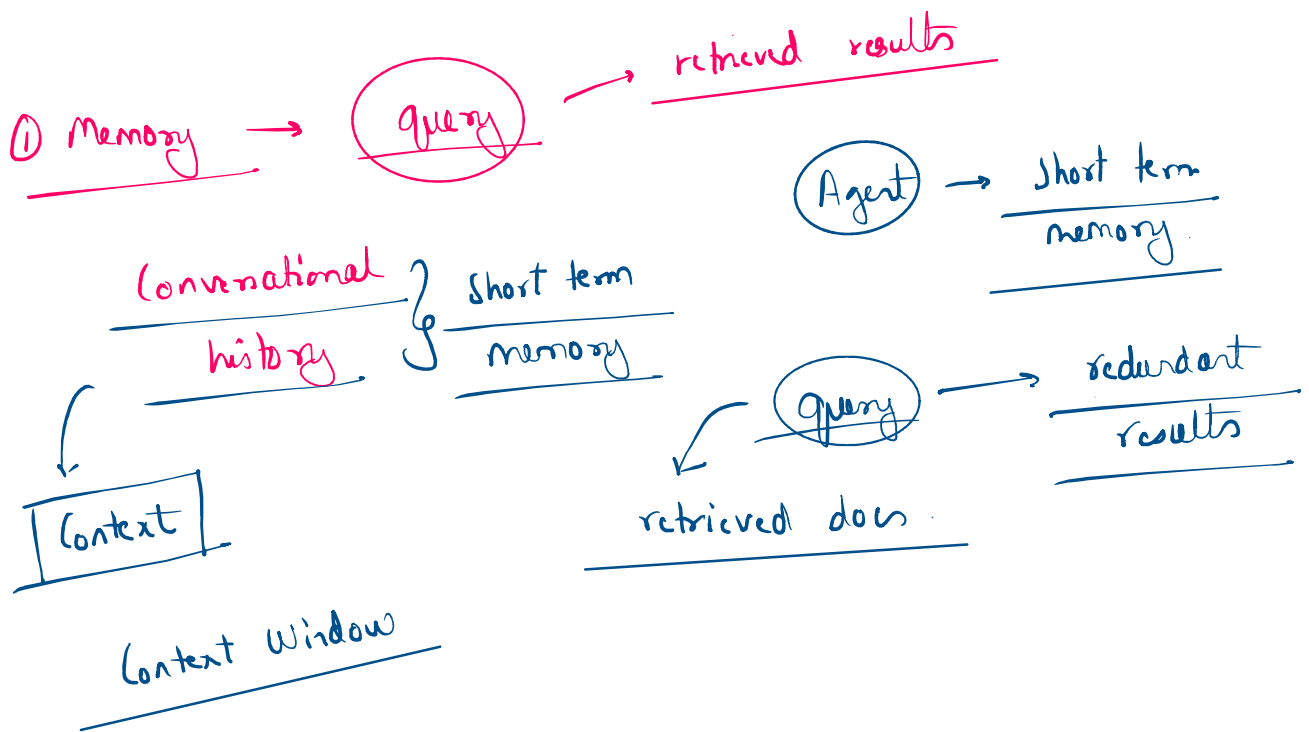
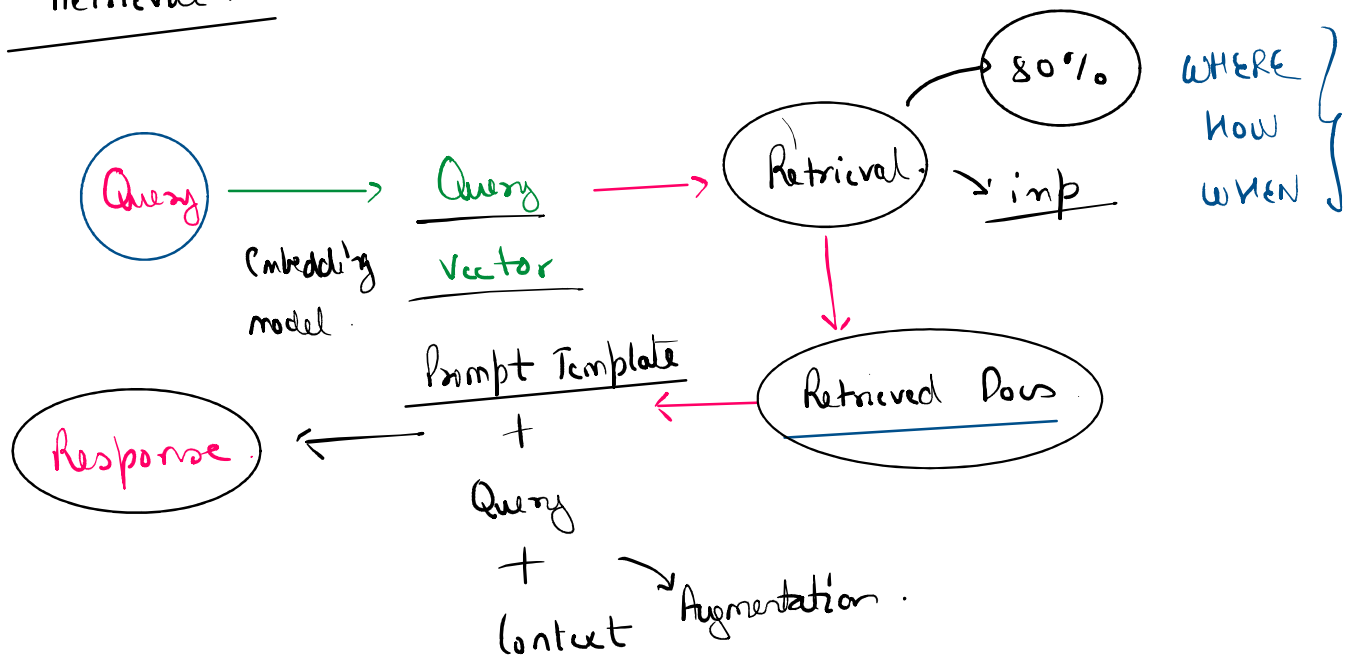
Retrieved

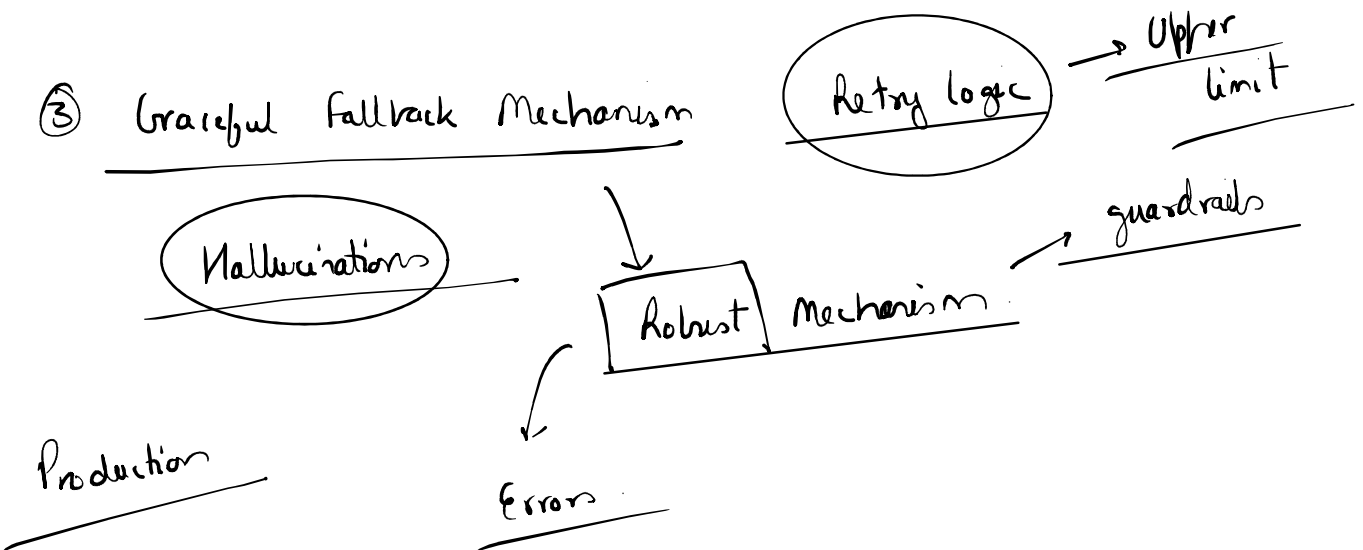
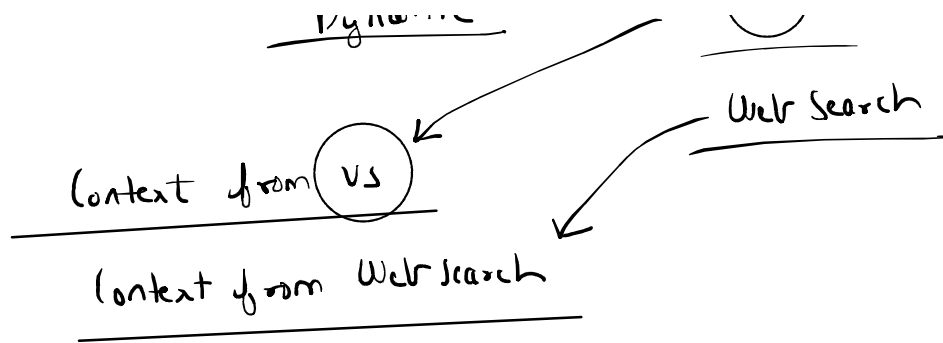
0.10 m



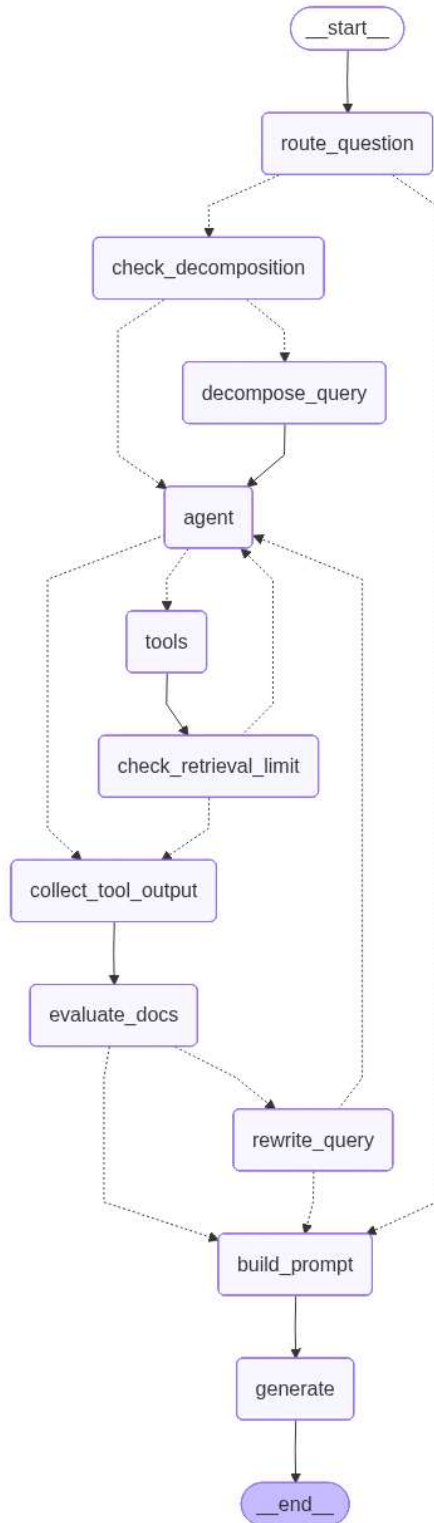
- ① Select Retrieval.
- ② Query Decomposition.
- ③ Query Rephrasing / Rewriting
- ④ Source of Retriever → Tools
- 5) How to do retrieval.
- 6) Have to perform any retrieval.
- 7) Relevancy check on retrieved docs.
- 8) Response → Verify
 - grounded
 - useful.

Retrieval.



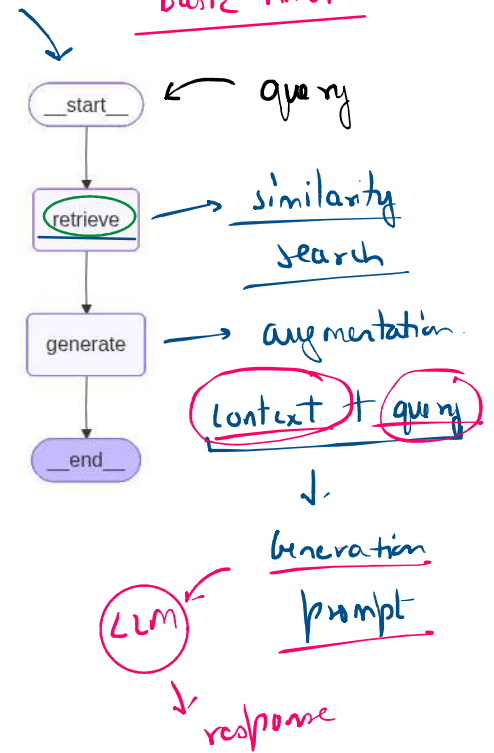


Agentic RAG



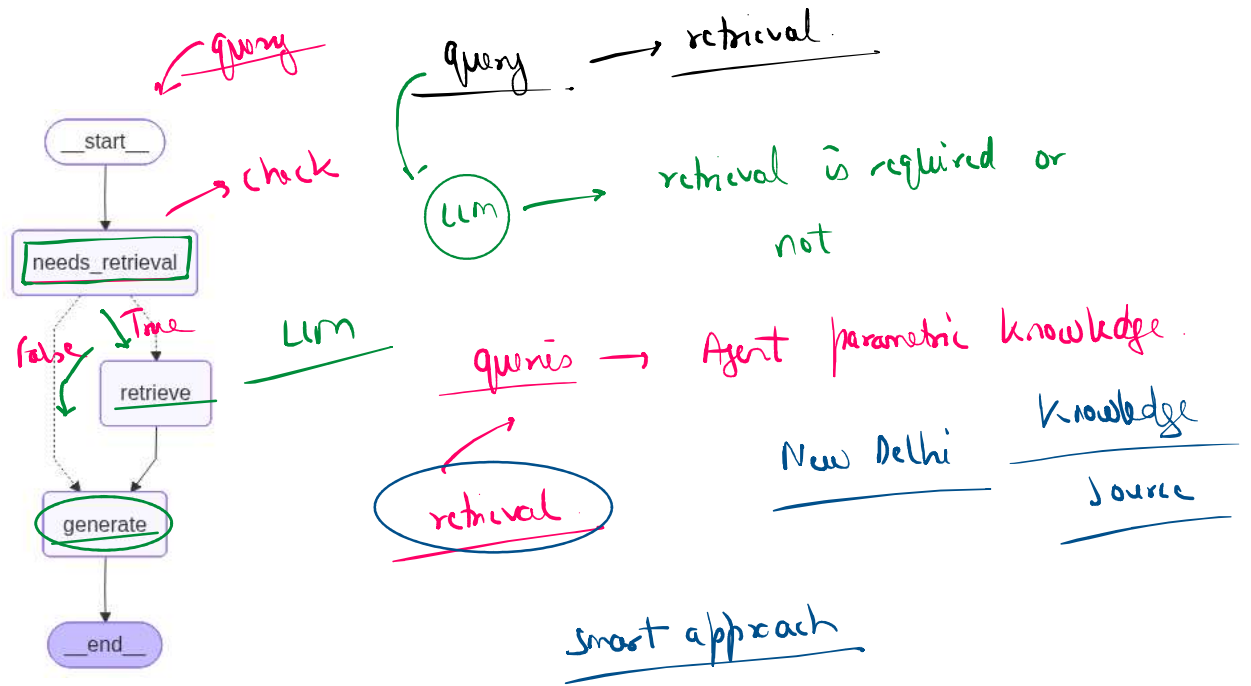
iterative.

Basic RAG

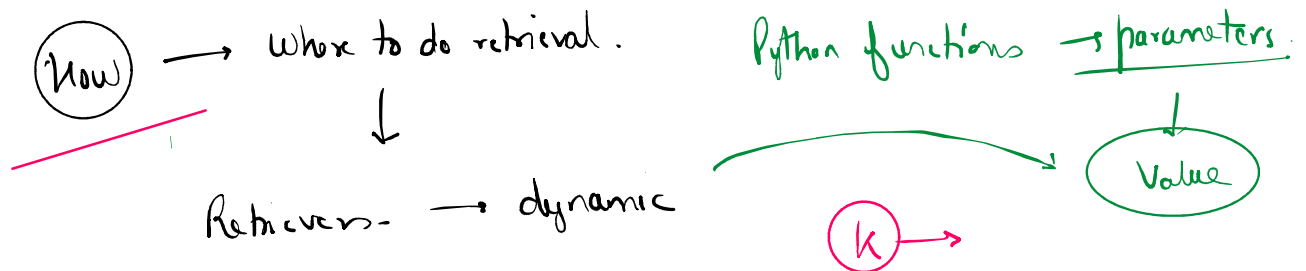
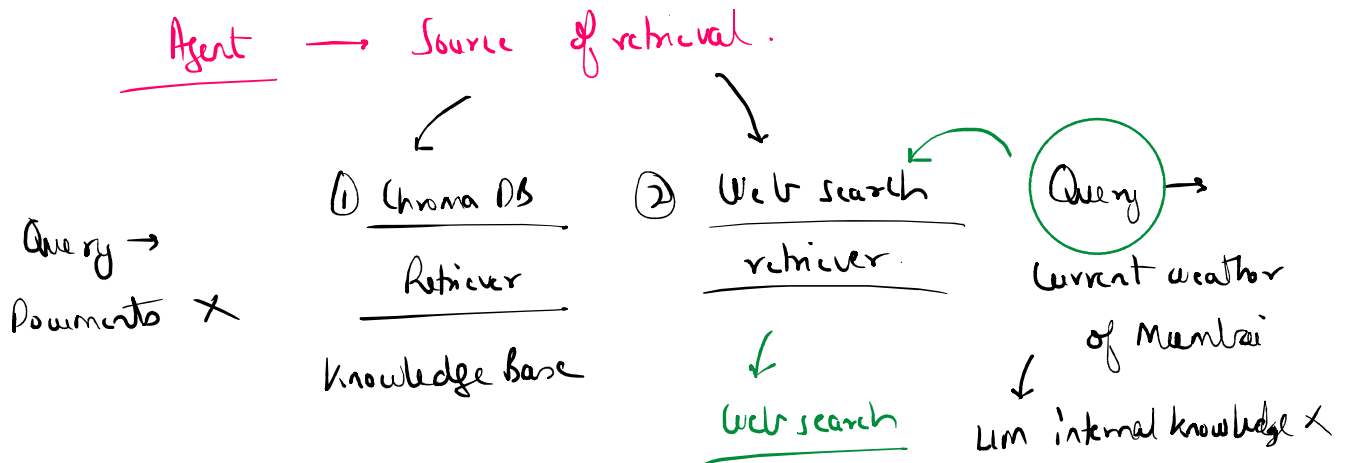
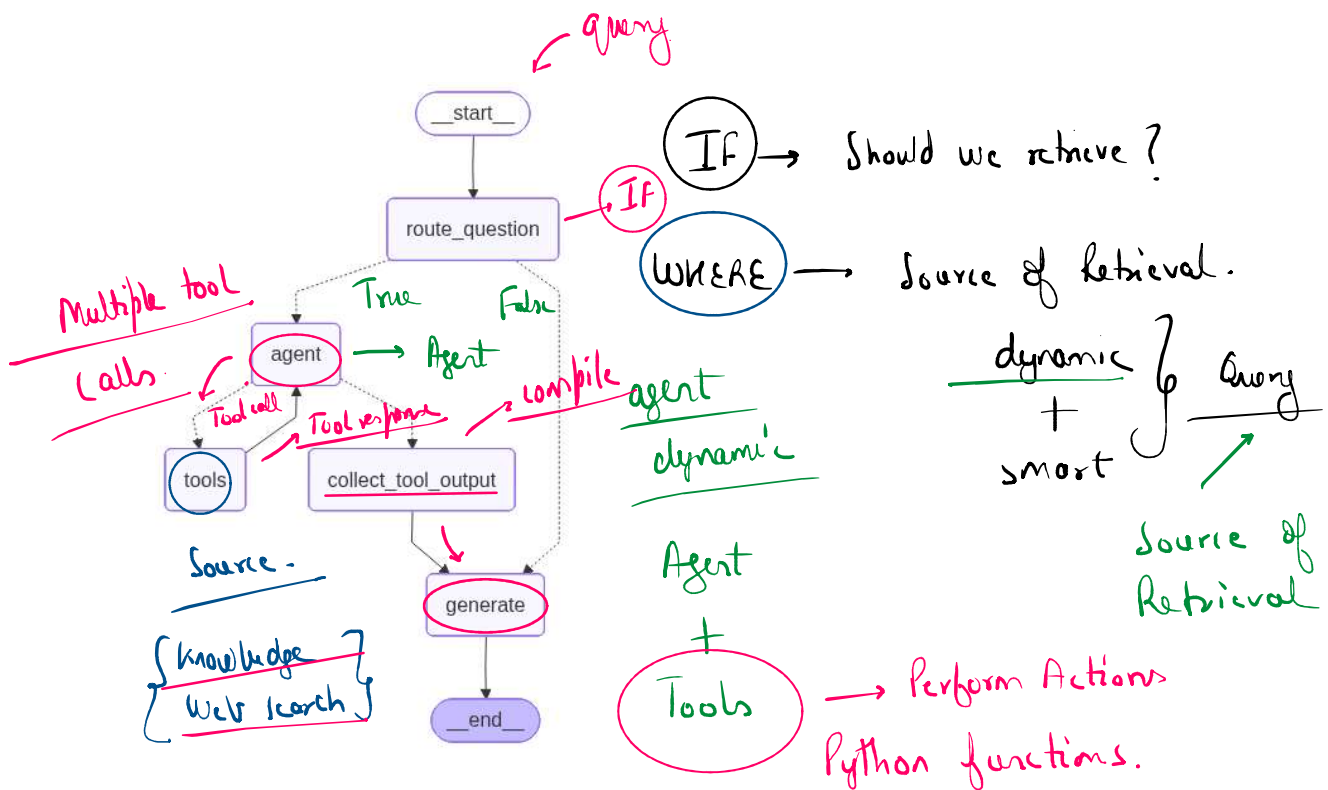


large graph

1 - Retrieval Decision

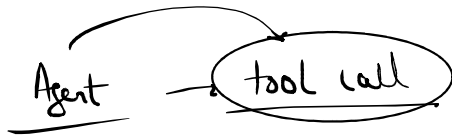
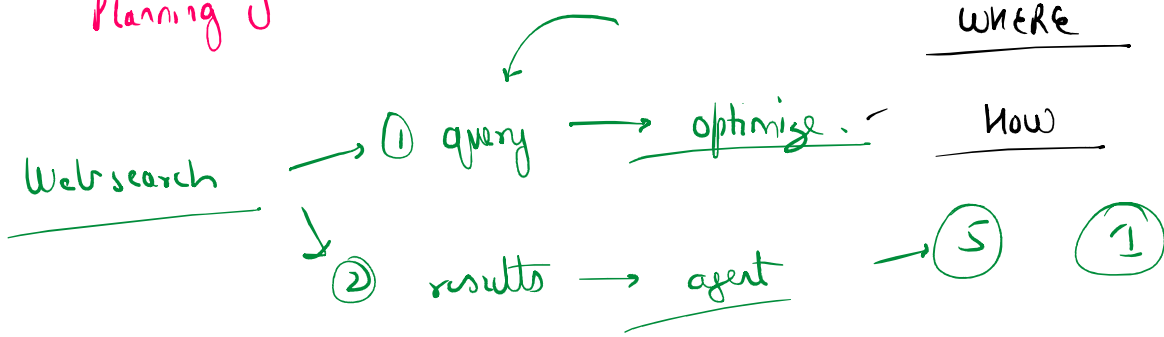


2 - Retrieval Source Selection



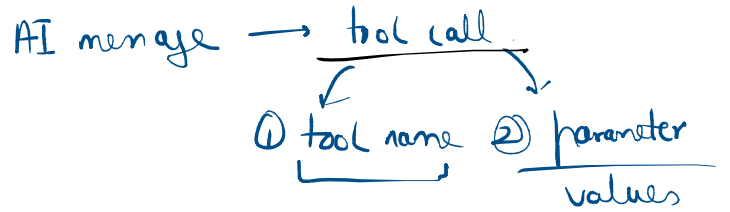
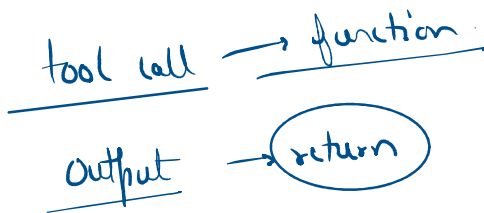
Reasoning / Iterative → Tool call operation.

Reasoning } cognitive → Tool call operation
 Planning }



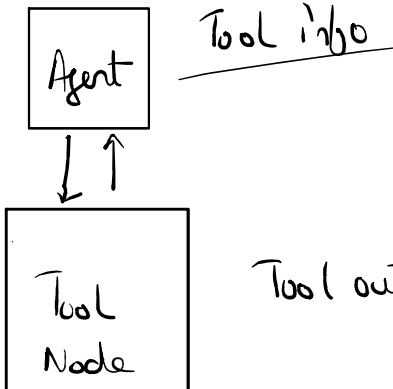
System message → Defines the persona, role

Human message → query



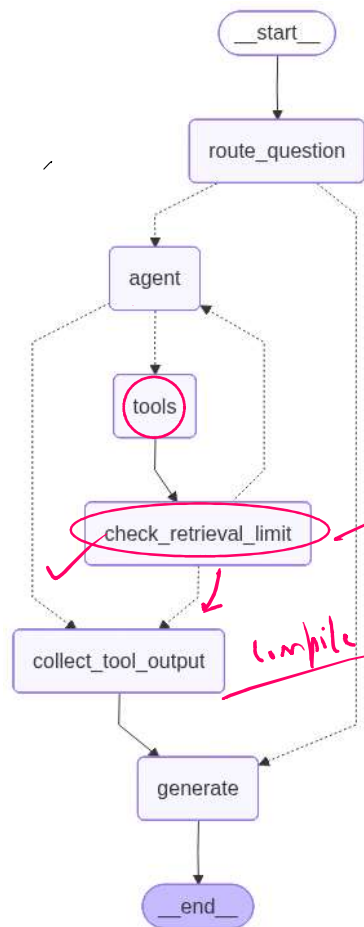
Tool message → Tool call output

AI message



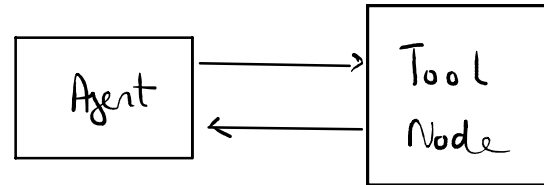
Tool output → tools define

3 - Retry Logic and Fallback



Query → complex

Agent → Retrieval loop.



Complex → Multiple tool calls.

Web search } → Parallel -
Retriever }

Tool call → AI Message

Agent → Multiple tool calls.

Loop

Tokens

3 Retriever

Retry logic

→ Tool Node

Fallback mechanism

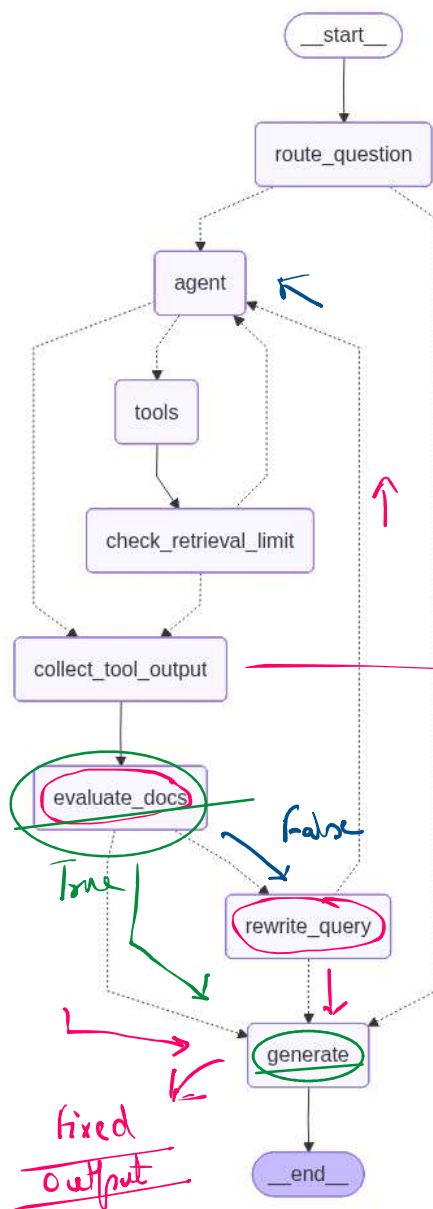
Maximum limit

Break the loop

tool calls

→ Break out
of the loop

4 - Relevancy Check and Query Rewriting



Relevancy check

Query rewriting

evaluate docs

retrieved docs

↓
compile

✓
relevancy check

for loop

D_1, D_2, D_3, D_4, D_5 → Relevancy check
+ query

relevant_docs

D_3, D_5
X X

relevant docs = []

query → rewrite → rephrase
loop

Breaking condition

2 Tasks

① Relevancy check

Noisy
discard.

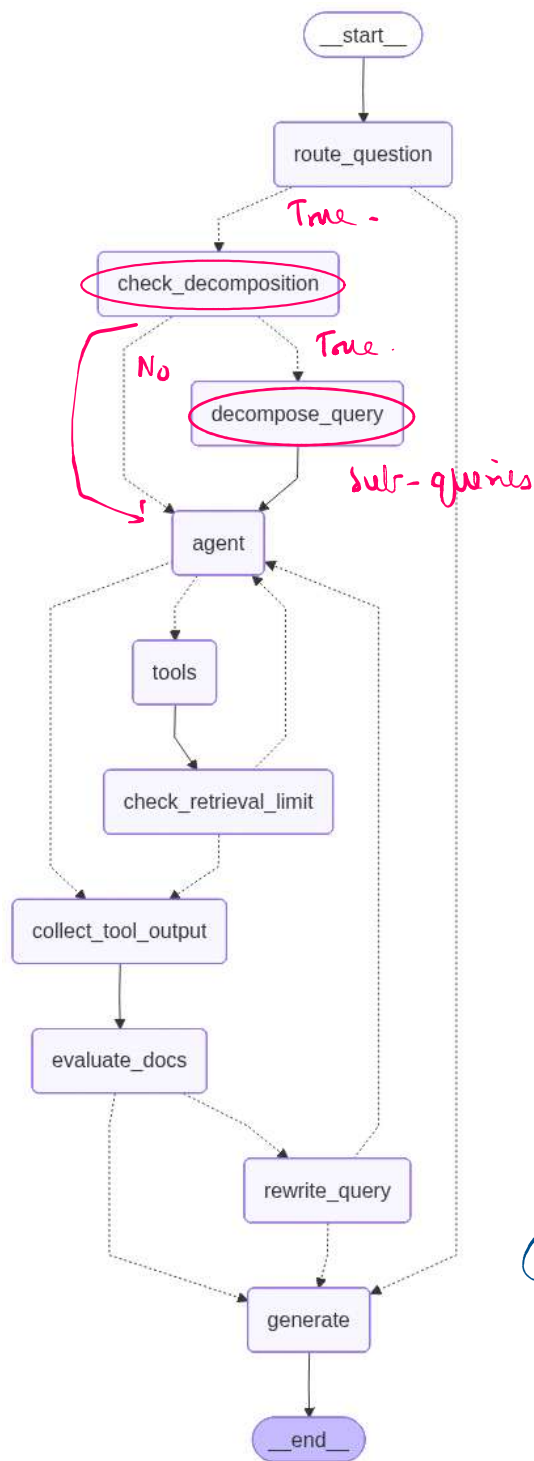
↓
Relevant docs → context

↓
Relevant docs → context

② Query → Rewrite / Rephrase.

Relevancy check.

5 - Query Decomposition



Query → complex

↓
Multiple Retrieval steps

Complex query

WHERE
HOW

Breakdown into
Simpler sub-queries → Retrieval

Check → Query original

Complex → Yes/No

Breakdown
into simple
sub-queries

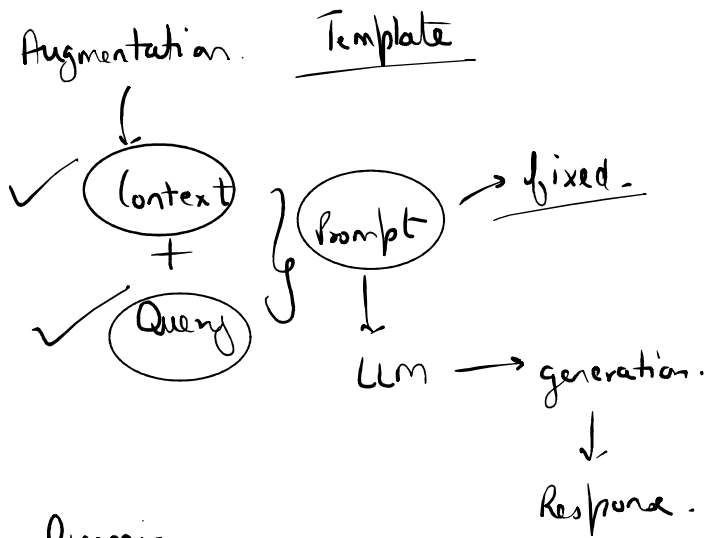
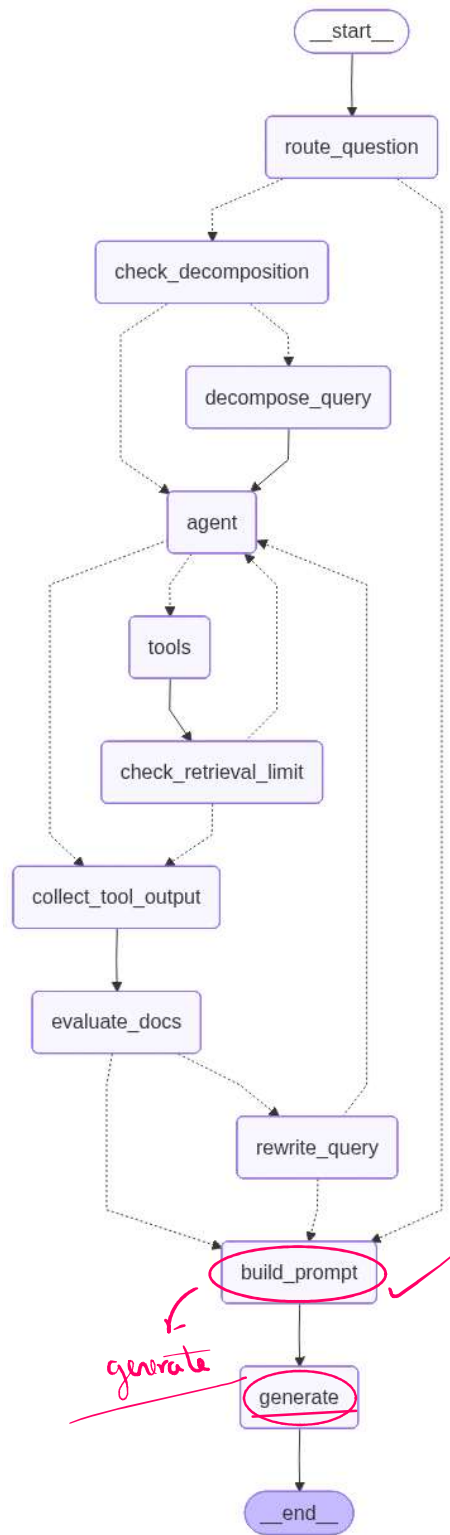
Use to do
retrieval.

② Steps

1) To check whether input query
is complex x

2) Decompose the query.

Agentic RAG



Prompt → Retrieval Sources.
Decompose.
better response → context information.

Context → Retrieved docs.
Query → Rewritten / decomposed.

Prompt → Context
Vs Web search

Information

Sources
information

generation → better

